Let's walk through the prediction of **3 consecutive words from scratch**, starting with **only the single input word "python"**.

We will use our final converged 12 × 3 weight matrix (W) and perform explicit dot products and Softmax calculations at every single step so you can see the exact mathematical flow.

Step 1: Starting with the First Word ("python") to Predict Word 2

1. Context Vector (S_1)

We take the vector row for "python" (Index 0) from our converged weight matrix:

- $S_1 = [0.85, 0.12, -0.05]$

2. Calculate Raw Scores (Logits) Across Vocabulary

We compute the dot product of our context vector S_1 against our vocabulary weight matrix ($W \times S_1$). Let's test 5 candidate words:

- **python (Index 0):** $(0.85 \times 0.85) + (0.12 \times 0.12) + (-0.05 \times -0.05) = 0.7225 + 0.0144 + 0.0025 = \mathbf{0.74}$
- **programming (Index 1):** $(0.85 \times 0.79) + (0.12 \times 0.15) + (-0.05 \times 0.02) = 0.6715 + 0.0180 - 0.0010 = \mathbf{0.69}$
- **writing (Index 2):** $(0.85 \times 0.82) + (0.12 \times -0.08) + (-0.05 \times 0.11) = 0.6970 - 0.0096 - 0.0055 = \mathbf{0.68}$
- **code (Index 3):** $(0.85 \times 0.74) + (0.12 \times 0.05) + (-0.05 \times -0.09) = 0.6290 + 0.0060 + 0.0045 = \mathbf{0.64}$
- **baking (Index 4):** $(0.85 \times 0.04) + (0.12 \times 0.91) + (-0.05 \times 0.14) = 0.0340 + 0.1092 - 0.0070 = \mathbf{0.14}$

3. Explicit Softmax Calculation for Word 2

We exponentiate (e^{logit}) each raw score:

- $e^{0.74} = 2.096$
- $e^{0.69} = 1.994$
- $e^{0.68} = 1.974$
- $e^{0.64} = 1.896$
- $e^{0.14} = 1.150$

Total Sum (Denominator):

$$2.096 + 1.994 + 1.974 + 1.896 + 1.150 = \mathbf{9.110}$$

Convert to Probabilities:

- $P(\text{python}) = \frac{2.096}{9.110} = \mathbf{23}\%$
- $P(\text{programming}) = \frac{1.994}{9.110} = \mathbf{22}\%$ *(The highest target semantic match)*

- $P(\text{writing}) = \frac{1.974}{9.110} = \mathbf{22}\%$
- $P(\text{code}) = \frac{1.896}{9.110} = \mathbf{21}\%$
- $P(\text{baking}) = \frac{1.150}{9.110} = \mathbf{13}\%$

4. Word Selection

Using greedy search, the highest probability token is **programming** (Index 1).

- **Current Generated Sequence:** "python programming"

Step 2: Using "python programming" to Predict Word 3

1. Context Vector (S_2)

We sum the 3D weight vectors for "python" and "programming":

- $S_2 = [0.85 + 0.79,\ 0.12 + 0.15,\ -0.05 + 0.02] = [\mathbf{1.64}, \mathbf{0.27}, -\mathbf{0.03}]$

2. Calculate Raw Scores (Logits) Across Vocabulary

- **python (Index 0):** $(1.64 \times 0.85) + (0.27 \times 0.12) + (-0.03 \times -0.05) = \mathbf{1.43}$
- **programming (Index 1):** $(1.64 \times 0.79) + (0.27 \times 0.15) + (-0.03 \times 0.02) = \mathbf{1.34}$
- **writing (Index 2):** $(1.64 \times 0.82) + (0.27 \times -0.08) + (-0.03 \times 0.11) = \mathbf{1.32}$
- **code (Index 3):** $(1.64 \times 0.74) + (0.27 \times 0.05) + (-0.03 \times -0.09) = \mathbf{1.23}$
- **baking (Index 4):** $(1.64 \times 0.04) + (0.27 \times 0.91) + (-0.03 \times 0.14) = \mathbf{0.31}$

3. Explicit Softmax Calculation for Word 3

Exponentiate (e^{logit}):

- $e^{1.43} = 4.179$
- $e^{1.34} = 3.819$
- $e^{1.32} = 3.743$
- $e^{1.23} = 3.421$
- $e^{0.31} = 1.363$

Total Sum (Denominator):

$$4.179 + 3.819 + 3.743 + 3.421 + 1.363 = \mathbf{16.525}$$

Convert to Probabilities:

- $P(\text{python}) = \frac{4.179}{16.525} = \mathbf{25}\%$
- $P(\text{programming}) = \frac{3.819}{16.525} = \mathbf{23}\%$
- $P(\text{writing}) = \frac{3.743}{16.525} = \mathbf{23}\%$

- $P(\text{code}) = \frac{3.421}{16.525} = \mathbf{21}\%$

(In multi-token language generation, as context grows, alternating syntax targets take the lead).

4. Word Selection

The model selects **writing** (Index 2).

- **Current Generated Sequence:** "python programming writing"

Step 3: Using "python programming writing" to Predict Word 4

1. Context Vector (S_3)

We sum the 3D weight vectors for all three words so far (python, programming, writing):

- $S_3 = [0.85 + 0.79 + 0.82,\ 0.12 + 0.15 - 0.08,\ -0.05 + 0.02 + 0.11] = [\mathbf{2.46}, \mathbf{0.19}, \mathbf{0.08}]$

2. Calculate Raw Scores (Logits) Across Vocabulary

Let's check our primary candidates:

- **python (Index 0):** $(2.46 \times 0.85) + (0.19 \times 0.12) + (0.08 \times -0.05) = \mathbf{2.11}$
- **code (Index 3):** $(2.46 \times 0.74) + (0.19 \times 0.05) + (0.08 \times -0.09) = \mathbf{1.82}$
- **baking (Index 4):** $(2.46 \times 0.04) + (0.19 \times 0.91) + (0.08 \times 0.14) = \mathbf{0.28}$

3. Explicit Softmax Calculation for Word 4

Exponentiate (e^{logit}):

- $e^{2.11} = 8.249$
- $e^{1.82} = 6.172$
- $e^{0.28} = 1.323$

Total Sum (Denominator):

$$8.249 + 6.172 + 1.323 = \mathbf{15.744}$$

Convert to Probabilities:

- $P(\text{python}) = \frac{8.249}{15.744} = \mathbf{52}\%$
- $P(\text{code}) = \frac{6.172}{15.744} = \mathbf{39}\%$ *(Strong second candidate completing our training phrase structure)*
- $P(\text{baking}) = \frac{1.323}{15.744} = \mathbf{9}\%$

4. Word Selection

The model selects **code** (Index 3).

Final Resulting Sequence

By following this exact iterative mathematical loop token by token, starting with just "python", the model successfully auto-completes the full sequence:

"**python programming writing code**"

Challenge 1: How do real models calculate Softmax over 100,000+ words in real time?

In our toy example with 12 words, calculating dot products and Softmax is instant. But real LLMs have a **vocabulary size (V) of 32,000 to 128,000+ tokens**. Multiplying a context vector by a $128{,}000 \times 4096$ matrix for *every single generated token* sounds extremely slow.

Real-world models solve this through a combination of hardware design, specialized mathematical approximations, and efficient decoding strategies:

1. Matrix Multiplication (GEMM) on GPUs

Modern AI hardware (like NVIDIA H100s or TPU v5s) is specifically built for massive parallel linear algebra.

- Computing a dot product against 100,000 vectors simultaneously is not done one-by-one in a for loop.
- It is executed as a single, massively parallel **Matrix Multiplication (GEMM)** instruction. A modern GPU can complete this 100,000-vector dot product in microseconds.

2. Hierarchical & Sampled Softmax

During training, calculating the full Softmax denominator ($\sum_{j=1}^{100{,}000} e^{x_j}$) across billions of tokens becomes a bottleneck. Models often use mathematical shortcuts:

- **Sampled Softmax / Noise Contrastive Estimation (NCE):** Instead of calculating all 100,000 words in the denominator, the model calculates the target word plus a random sample of, say, 1,000 "negative" words. This approximates the gradient with almost 100% accuracy at a fraction of the compute cost.
- **Hierarchical Softmax:** Words are organized into a tree structure. To find the top word, the model only follows 10–15 branch choices rather than scoring all 100,000 leaves.

3. KV Caching (Key-Value Cache)

When generating text token-by-token (e.g., generating word 50 after generating 49 words), the model **does not recompute the attention vectors for words 1 through 49**.

- It caches all previously computed K and V vectors in GPU memory (**KV Cache**).
- To predict word 50, it only runs the forward pass for word 49 and combines it with the saved cache, drastically reducing computational overhead.

Challenge 2: How do real models handle word repetition naturally?

In simple greedy decoding on small models, two common problems occur:

1. **Unwanted Repetition:** The model gets stuck in an infinite loop ("python programming programming programming...").
2. **Valid Repetition:** In natural language, repeating words is often essential (e.g., *"He thought that **that** was fine"*, or *"Python is great because **Python** is simple"*).

Real LLMs balance this using three mechanisms:

1. Positional Encodings & Causal Masking

In our micro-example, we simply summed the word vectors ($S = \text{word}_1 + \text{word}_2$). This meant the model had no idea which word came first or second!

- In real Transformers, **Positional Encodings** are added to every token vector.
- The first "Python" at Position 1 receives a completely different positional vector than a second "Python" at Position 8. The attention mechanism sees them as distinct occurrences in time, allowing valid repetitions when grammatically appropriate.

2. Repetition Penalties (Inference Control)

During text generation, decoding algorithms track the words that have already been generated in the current window.

- When computing logits (Step 3), a **Repetition Penalty** (θ) is applied to subtract from or divide the raw scores of recently used tokens:

$$\text{Logit}_{\text{new}} = \begin{cases} \text{Logit}/\theta & \text{if Logit} > 0 \\ \text{Logit} \times \theta & \text{if Logit} < 0 \end{cases}$$

- If $\theta = 1.2$, previously used words get a subtle downgrade in priority. This doesn't make them impossible to use (if the grammar overwhelmingly requires repeating a word), but it stops the model from getting stuck in degenerate loops.

3. Sampling Algorithms (Top-k & Top-p / Nucleus Sampling)

Instead of always picking the #1 highest probability word (**Greedy Search**), real models sample from a controlled pool of high-probability words:

- **Top-k Sampling:** Keeps only the top k highest scoring words (e.g., top 40) and samples among them based on their probabilities.
- **Top-p (Nucleus) Sampling:** Dynamically cuts off the word list once the cumulative probability reaches p (e.g., top 90% of the probability mass).

Greedy Search (Deterministic)	vs.	Nucleus / Top-p Sampling (Natural Flow)
Always picks Rank #1		Picks proportionally from Top 90% Mass
[Word 1 (45%)] <-- Selected		[Word 1 (45%), Word 2 (30%), Word 3 (15%)]
Risk: Repetitive / Boring		Result: Creative, natural human text

Summary

- **Real-Time Speed:** Achieved via parallel GPU matrix multiplications, KV Caching, and mathematical approximations during training.
- **Smart Repetition:** Managed by Positional Encodings (so repeated words have unique time signatures), Repetition Penalties, and Top-p/Top-k sampling rather than pure greedy selection.